Deep Learning with PyTorch & CNN

Complete Technical Reference: Libraries, Functions, Control Flow & AI Concepts

Notebook: Feature Extraction and Image Classification with CNN (CIFAR-10)

1. Libraries Used

1.1 torch (PyTorch Core)

PyTorch is an open-source machine learning framework developed by Facebook AI Research. It provides a dynamic computational graph (eager execution), meaning operations execute immediately — unlike TensorFlow 1.x which required building a static graph first. This makes debugging natural and Python-idiomatic.

Key capabilities:

- N-dimensional tensor operations on CPU and GPU
- Automatic differentiation via torch.autograd
- A neural network module system (torch.nn)
- Optimizers, loss functions, data utilities

1.2 torchvision

torchvision is an official companion library to PyTorch for computer vision tasks. It provides:

- Pre-built datasets (CIFAR-10, ImageNet, MNIST, etc.)
- Pre-trained model architectures (ResNet, VGG, etc.)
- Image transformation and augmentation utilities (torchvision.transforms)
- Utility functions like make_grid for visualization

1.3 torchvision.transforms

A sub-module of torchvision providing composable image preprocessing and augmentation pipelines. Transforms are applied lazily via a Compose chain, making it easy to build complex data pipelines. Used to convert raw PIL images to tensors and normalize pixel values before feeding them to the network.

1.4 matplotlib.pyplot

matplotlib is Python's foundational 2D plotting library. The pyplot interface provides a MATLAB-style stateful API for creating figures, axes, and plots. In this notebook it is used to render image grids from the dataset so the user can visually inspect training samples.

1.5 numpy

NumPy is the fundamental package for scientific computing in Python. It provides the ndarray (n-dimensional array) type with efficient C-backed operations. PyTorch tensors can be converted to NumPy arrays (and vice versa) for operations not natively supported in PyTorch, such as reshaping label arrays or stacking feature matrices.

1.6 torch.nn

torch.nn is PyTorch's neural network module. It provides:

- nn.Module — the base class every custom network inherits from
- Layer types: Conv2d, Linear, ReLU, MaxPool2d, Dropout, BatchNorm2d, etc.
- Loss functions: CrossEntropyLoss, MSELoss, BCELoss, etc.
- Sequential and other container modules

1.7 torch.nn.functional (F)

This module contains the functional (stateless) versions of all nn layers. While nn.Conv2d stores weights as parameters, F.conv2d takes weights as explicit arguments. Used in this notebook (though commented out) for F.softmax — applying softmax to convert raw logits to probabilities.

1.8 torch.optim

torch.optim provides standard optimization algorithms. It manages parameter updates during training by reading gradients computed by autograd and applying update rules. In this notebook, SGD (Stochastic Gradient Descent) with momentum is used.

1.9 sklearn.ensemble (scikit-learn)

scikit-learn is Python's standard machine learning library for classical (non-deep learning) algorithms. The ensemble sub-module provides ensemble methods — algorithms that combine multiple weak learners. RandomForestClassifier from this module is used as the final classifier on top of CNN-extracted features.

1.10 joblib

joblib is a lightweight Python library for serialization and parallel computing. In this notebook it is used to save and load the trained scikit-learn Random Forest model to disk (.pkl files), enabling model persistence without retraining.

2. Functions — Explained Line by Line

2.1 Data Loading & Preprocessing

transforms.Compose([...])

Chains multiple transform operations into a single callable pipeline. Each transform in the list is applied sequentially to every image before it reaches the network. The list is evaluated left to right: ToTensor first, then Normalize.

```
transforms.Compose([transforms.ToTensor(), transforms.Normalize(...)])
```

transforms.ToTensor()

Converts a PIL Image (H x W x C, uint8, range 0-255) to a PyTorch float tensor (C x H x W, float32, range 0.0-1.0). Also transposes the channel dimension from last to first to match PyTorch's NCHW convention (batch, channels, height, width).

transforms.Normalize(mean, std)

Normalizes each channel of a tensor image using the formula: output = (input - mean) / std. With mean=0.5 and std=0.5 for all three channels, the pixel values shift from [0,1] to [-1,1]. Normalization stabilizes gradient flow during training by centering the data distribution around zero.

torchvision.datasets.CIFAR10(...)

Downloads (if not already present) and loads the CIFAR-10 dataset. Arguments:

- root='./data' — directory to store data
- train=True/False — selects training or test split
- download=True — fetches from internet if missing
- transform=transform — applies the preprocessing pipeline to each image

Returns a Dataset object (not the data itself — it is loaded lazily).

torch.utils.data.DataLoader(dataset, batch_size, shuffle, num_workers)

Wraps a Dataset and provides an iterable that yields batches. Key arguments:

- batch_size=4 — number of samples per batch
- shuffle=True — randomizes sample order each epoch (training only; disabled for test)
- num_workers=2 — number of parallel CPU worker processes for data loading

Internally uses a Sampler to determine iteration order and a collate_fn to stack individual samples into batched tensors.

iter(trainloader) and next(dataiter)

iter() converts the DataLoader into a Python iterator. next() retrieves the next batch as a tuple of (images_tensor, labels_tensor). Used here for visualizing a single batch before training.

2.2 Visualization Functions

imshow(img) — custom function

Defined in the notebook to display a PyTorch image tensor:

- img / 2 + 0.5 — unnormalizes from [-1,1] back to [0,1] (reverses Normalize)
- img.numpy() — converts tensor to NumPy array
- np.transpose(npimg, (1, 2, 0)) — reorders from (C, H, W) to (H, W, C) for matplotlib
- plt.imshow(...) — renders the image
- plt.show() — displays the figure window

torchvision.utils.make_grid(images)

Arranges a batch of image tensors into a single grid image. Useful for visualizing multiple images at once. Adds padding between images. Returns a single tensor (3, H, W) that can be passed to imshow.

2.3 Neural Network Definition

class Net(nn.Module) — model class

Defines the CNN architecture by inheriting from nn.Module, PyTorch's base class for all neural networks. Every custom network must:

- Call super().__init__() in __init__ to register PyTorch internals
- Define layers as instance attributes in __init__
- Implement forward(self, x) to define the computation graph

nn.Conv2d(in_channels, out_channels, kernel_size)

A 2D convolutional layer. Applies a set of learnable filters over the input image. Each filter slides across the spatial dimensions, computing dot products to produce a feature map.

- self.conv1 = nn.Conv2d(3, 6, 5) — 3 input channels (RGB), 6 filters, 5x5 kernel
- self.conv2 = nn.Conv2d(6, 16, 5) — 6 input channels, 16 filters, 5x5 kernel

Output spatial size: floor((input_size - kernel_size) / stride + 1). Without padding, a 32x32 input after a 5x5 conv becomes 28x28.

nn.ReLU()

Rectified Linear Unit activation function. Applies max(0, x) element-wise. Introduces non-linearity into the network, enabling it to learn complex mappings. ReLU is preferred over sigmoid/tanh in deep networks because it does not saturate for positive values, mitigating the vanishing gradient problem.

nn.MaxPool2d(kernel_size, stride)

Max pooling layer. Divides the input into non-overlapping windows and takes the maximum value from each. With kernel_size=2 and stride=2, it halves the spatial dimensions (downsampling), reducing computation and providing spatial invariance to small translations.

nn.Linear(in_features, out_features)

A fully connected (dense) layer. Every input neuron is connected to every output neuron via a learned weight matrix. Computes output = input @ weight.T + bias. Three FC layers are stacked:

- self.fc1 = nn.Linear(16*5*5, 120) — flattened conv output to 120 neurons
- self.fc2 = nn.Linear(120, 84) — 120 to 84 neurons
- self.fc3 = nn.Linear(84, 10) — 84 to 10 class scores (one per CIFAR-10 class)

x.view(-1, 16 * 5 * 5) — tensor reshaping

Reshapes a multi-dimensional tensor into a 1D vector per sample for input to the fully connected layers. -1 tells PyTorch to infer the batch dimension automatically. This is the 'flatten' operation between convolutional and linear layers.

super(Net, self).__init__()

Calls the parent class (nn.Module) constructor. Required by PyTorch to properly register parameters, buffers, and hooks. Without it, PyTorch's internal machinery (gradient tracking, device management, state_dict) will not work correctly.

2.4 Loss Function and Optimizer

nn.CrossEntropyLoss()

Computes the cross-entropy loss between predicted class scores (logits) and ground truth class indices. Internally combines LogSoftmax and NLLLoss. Appropriate for multi-class classification. The loss is minimized when the model assigns high probability to the correct class.

optim.SGD(net.parameters(), lr, momentum)

Stochastic Gradient Descent optimizer with momentum.

- net.parameters() — returns an iterator over all learnable parameters in the model
- lr=0.001 — learning rate: controls the step size of each parameter update
- momentum=0.9 — accumulates a velocity vector in the gradient direction, helping navigate flat regions and dampen oscillations

Update rule: velocity = momentum * velocity - lr * gradient; param += velocity

2.5 Training Loop Functions

optimizer.zero_grad()

Clears (sets to zero) the gradients of all parameters tracked by the optimizer. PyTorch accumulates gradients by default (adds to existing .grad). This must be called before each forward pass to prevent gradients from previous batches from being added to the current batch's gradients.

net(inputs) — forward pass

Calls the network's forward() method. In this notebook, forward() returns a tuple of three tensors: (x3, x2, x) corresponding to the flattened conv features (400-dim), the fc2 output (84-dim), and the final class scores (10-dim). This design enables feature extraction from intermediate layers.

criterion(outputs, labels) — loss computation

Evaluates the CrossEntropyLoss between the model's predicted logits (outputs) and the integer class labels. Returns a scalar tensor representing the average loss over the batch.

loss.backward()

Triggers reverse-mode automatic differentiation (backpropagation). PyTorch traverses the computational graph in reverse, computing gradients of the loss with respect to every parameter using the chain rule. Gradients are accumulated in each parameter's .grad attribute.

optimizer.step()

Updates all parameters according to the optimizer's update rule using the gradients stored in .grad. For SGD with momentum, it applies the velocity-based update. This is the actual learning step.

loss.item()

Extracts the scalar value from a single-element tensor as a Python float. Used to accumulate running loss for printing statistics. Calling .item() detaches the value from the computational graph, preventing memory leaks.

running_loss += loss.item() and periodic print

Accumulates the loss over 2000 mini-batches, then prints the average and resets to zero. This provides a smoothed loss curve without printing every single batch.

2.6 GPU Support

torch.cuda.is_available()

Returns True if a CUDA-capable GPU is detected and CUDA drivers are installed. Used as a flag to conditionally move tensors and model to GPU.

net.cuda()

Moves all model parameters and buffers to the default GPU (cuda:0). After this call, all computations involving the model occur on the GPU. Equivalent to net.to('cuda').

inputs.cuda() / labels.cuda()

Moves input tensors to the GPU. Both model and data must reside on the same device for computation to proceed. If one is on CPU and the other on GPU, PyTorch raises a RuntimeError.

features.cpu()

Moves a tensor back to CPU memory. Required before converting to NumPy, since NumPy does not support CUDA tensors. Also needed before passing data to scikit-learn which operates on CPU.

2.7 Model Persistence

torch.save(net.state_dict(), path)

Serializes the model's state_dict (an OrderedDict mapping parameter names to their tensor values) to a file. state_dict() contains only parameters and buffers — not the model architecture. The file format is Python's pickle.

net.load_state_dict(torch.load(path))

Loads saved weights back into the model. torch.load() deserializes the file; load_state_dict() copies the tensors into the model's parameters. The model architecture must match the saved state_dict exactly.

net.eval()

Switches the model to evaluation mode. This affects layers that behave differently during training vs inference:

- Dropout: disabled (no random zeroing) in eval mode
- BatchNorm: uses running statistics instead of batch statistics

Does NOT disable gradient computation — that requires torch.no_grad() context manager.

2.8 Inference & Evaluation Functions

torch.max(tensor, dim)

Returns the maximum values and their indices along a given dimension. With dim=1, it finds the highest-scoring class for each sample in a batch. The returned indices are the predicted class labels. Returns a named tuple (values, indices).

```
_, predicted = torch.max(outputs.data, 1)
```

outputs.data

Accesses the raw data tensor of a tensor without the gradient tracking metadata. Older PyTorch idiom (before torch.no_grad() was common) used to prevent accidental gradient computation during inference.

predicted.cpu().numpy()

Two-step conversion: moves tensor to CPU first (.cpu()), then converts to NumPy ndarray (.numpy()). The resulting array can be used for indexing Python lists (e.g., looking up class names) or passed to scikit-learn functions.

labels.size(0)

Returns the size of the first dimension of the labels tensor — i.e., the batch size. Used to accumulate total sample count when computing accuracy across all batches.

(predicted == labels.cpu()).sum()

Element-wise comparison returning a boolean tensor, then .sum() counts True values — the number of correct predictions in a batch. Accumulated across batches gives total correct predictions.

(predicted == labels.cpu()).squeeze()

.squeeze() removes dimensions of size 1. For a batch of size 4, the comparison yields shape (4,). If shape is (4,1), squeeze makes it (4,) for clean indexing per sample.

2.9 Feature Extraction Functions

_, features, _ = net(inputs) — unpacking tuple

Uses Python tuple unpacking with _ as a discard variable to extract only the intermediate fc2 features (84-dim) from the network's 3-output forward(). The first _ is the conv features (400-dim) and the last _ is the class scores (10-dim). This is the core of the CNN-as-feature-extractor paradigm.

features.data.numpy()

Converts the feature tensor to a NumPy array for use with scikit-learn. .data removes the autograd wrapper; .numpy() does the actual conversion.

np.reshape(label, (labels.size(0), 1))

Reshapes the label array from shape (N,) to (N, 1). This makes it a column vector, compatible with np.vstack when building the full label matrix over all batches.

np.copy(feature)

Creates a deep copy of the NumPy array. Used for the first batch to initialize the accumulation matrices (featureMatrix, labelVector) without aliasing.

np.vstack([featureMatrix, feature])

Vertically stacks NumPy arrays by concatenating along the first axis (rows). Accumulates per-batch feature arrays into a single matrix over the entire dataset. After all batches, featureMatrix has shape (N_total, 84).

2.10 Random Forest Functions

RandomForestClassifier(n_estimators=100)

Creates a Random Forest with 100 decision trees. Each tree is trained on a bootstrap sample (random subset with replacement) of the training data, and each split considers only a random subset of features. Final prediction is by majority vote.

clf.get_params()

Returns a dictionary of all hyperparameters set on the classifier (n_estimators, max_depth, criterion, etc.). Useful for inspecting defaults and reproducibility.

clf.fit(featureMatrix, np.ravel(labelVector))

Trains the Random Forest on the CNN-extracted features. np.ravel() flattens the (N, 1) column-vector label array to a 1D array (N,) as required by scikit-learn's fit interface.

clf.predict(featureMatrixTest)

Runs inference on the test feature matrix. Each tree votes on the class; the majority class is returned. Returns an array of integer class indices with shape (N_test,).

clf.feature_importances_

An array of shape (n_features,) giving the relative importance of each feature (each of the 84 CNN neurons) as computed by the Random Forest. Importance is the mean and standard deviation of impurity decrease across all trees for each feature.

joblib.dump(clf, path) and joblib.load(path)

dump() serializes the trained scikit-learn model to a .pkl file using efficient binary serialization. load() deserializes it back to a Python object. Functionally similar to pickle but more efficient for large NumPy arrays within models.

np.ravel(labelVectorTest)

Flattens a multi-dimensional array to 1D. Used here to convert the (N, 1) test label matrix back to a (N,) array for element-wise comparison with predicted labels.

3. Model Control Flow

The notebook implements a complete ML pipeline with two stages: deep feature extraction via CNN and classification via Random Forest. Here is the full control flow:

3.1 Data Pipeline

Raw CIFAR-10 images (32x32 RGB, 10 classes, 50,000 train / 10,000 test) are downloaded. A preprocessing pipeline is composed: each PIL image is converted to a (3, 32, 32) float tensor by ToTensor, then each channel is normalized from [0,1] to [-1,1] by Normalize. DataLoaders wrap the datasets, yielding shuffled batches of 4 images during training and ordered batches during testing.

3.2 CNN Forward Pass

For each input batch x of shape (4, 3, 32, 32):

- x = conv1(x) → (4, 6, 28, 28) [3→6 channels, 5x5 conv, no padding]
- x = relu1(x) → (4, 6, 28, 28) [non-linearity]
- x = pool1(x) → (4, 6, 14, 14) [2x2 max-pool, halves spatial dims]
- x = conv2(x) → (4, 16, 10, 10) [6→16 channels, 5x5 conv]
- x = relu2(x) → (4, 16, 10, 10) [non-linearity]
- x = pool2(x) → (4, 16, 5, 5) [2x2 max-pool]
- x3 = x.view(-1, 400) → (4, 400) [flatten, saved as feature1]
- x = fc1(x3) → (4, 120) [fully connected]
- x = relu3(x) → (4, 120)
- x2 = fc2(x) → (4, 84) [saved as features — the feature vector]
- x = relu4(x2) → (4, 84)
- x = fc3(x) → (4, 10) [class logits]
- return (x3, x2, x) [all three intermediate outputs]

3.3 Training Loop

For each epoch (1 epoch in this notebook):

- Iterate over all batches in trainloader
- Move inputs and labels to GPU if available
- zero_grad() — clear accumulated gradients
- Forward pass — get (feature1, features, outputs)
- Compute CrossEntropyLoss on the 10-class outputs vs. ground truth labels
- backward() — compute gradients via backpropagation
- optimizer.step() — update weights with SGD + momentum
- Accumulate running loss; print every 2000 batches

3.4 Evaluation (CNN Classifier)

- Set net.eval() — disable dropout/batchnorm training behavior
- Iterate testloader without gradient tracking
- Forward pass — get (_, _, outputs), take torch.max for predicted class
- Compare predicted vs. true labels; count correct predictions
- Report overall and per-class accuracy

3.5 Feature Extraction

- Run the trained CNN in inference mode over all train and test batches
- Extract only x2 (84-dim fc2 output) from the forward pass tuple
- Move features/labels to CPU, convert to NumPy
- Accumulate into featureMatrix (50000, 84) and labelVector (50000, 1) for train
- Same process for test: featureMatrixTest (10000, 84)

3.6 Random Forest Classification

- RandomForestClassifier(n_estimators=100) — 100 trees
- clf.fit(featureMatrix, labels) — train forest on CNN features
- clf.predict(featureMatrixTest) — classify test features
- Compare predicted vs. true labels; compute overall and per-class accuracy
- clf.feature_importances_ — identify which CNN neurons are most discriminative
- Save model with joblib.dump()

4. AI Concepts Explained

4.1 Convolutional Neural Network (CNN)

A CNN is a deep learning architecture designed for processing grid-structured data such as images. Unlike fully connected networks that treat all pixels as independent features, CNNs exploit spatial locality and translation equivariance through shared-weight convolutional filters. The key insight is that the same visual feature (an edge, a curve) can appear anywhere in an image, so the same filter can detect it regardless of position.
CNNs are organized in hierarchical stages: early layers detect low-level features (edges, textures), middle layers detect patterns (shapes, object parts), and later layers detect high-level semantic features (objects, faces). This hierarchy is an emergent property of training, not manually designed.

4.2 Convolution Operation

Convolution slides a small learnable filter (kernel) across the input, computing the dot product at each position. This produces a feature map that encodes where a particular pattern was found.

With a 5x5 kernel and 6 filters, the first conv layer learns 6 different spatial detectors, each responding to different local image patterns.
Key properties: weight sharing (same filter applied everywhere reduces parameters vs. FC), local connectivity (each neuron sees only a local patch), and equivariance (shifting the input shifts the output correspondingly).

4.3 Max Pooling

Max pooling reduces spatial dimensions by taking the maximum value in each local window. It introduces a degree of spatial invariance: small shifts in feature position do not change the pooled output. It also reduces the number of parameters and computation in subsequent layers, helping prevent overfitting and speeding up training.

4.4 Activation Functions (ReLU)

Without non-linear activation functions, stacking multiple linear layers would still produce only a linear transformation. Activation functions introduce non-linearity that allows the network to approximate any continuous function (universal approximation theorem).
ReLU (Rectified Linear Unit) f(x) = max(0, x) is the default choice in modern CNNs because: it is computationally cheap, it does not saturate for positive values (solving the vanishing gradient problem that plagues sigmoid/tanh in deep networks), and it induces sparsity (inactive neurons produce exactly zero).

4.5 Backpropagation

Backpropagation is the algorithm for computing gradients of a scalar loss with respect to all parameters in a neural network. It applies the chain rule of calculus recursively, working backwards from the loss through each layer. In PyTorch, this is handled automatically by the autograd engine, which builds a dynamic computational graph during the forward pass and traverses it in reverse during loss.backward().

4.6 Stochastic Gradient Descent (SGD) with Momentum

Gradient descent updates parameters in the direction that locally decreases the loss. Stochastic refers to using a mini-batch rather than the full dataset for each update step, which introduces noise but dramatically reduces computation per step and often helps escape local minima.
Momentum adds a velocity term that accumulates gradient history, accelerating movement in consistent gradient directions while dampening oscillations in directions with conflicting gradients. With momentum=0.9, 90% of the previous velocity is retained and 10% of the new gradient is added each step.

4.7 Cross-Entropy Loss

Cross-entropy loss measures the dissimilarity between the true label distribution (a one-hot vector) and the predicted probability distribution (from softmax). For multi-class classification: L

= -log(p_correct), where p_correct is the predicted probability of the ground-truth class. Minimizing this loss pushes the network to assign high probability to the correct class. PyTorch's CrossEntropyLoss takes raw logits (pre-softmax scores) and integer class indices, combining softmax and log-loss into a single numerically stable operation.

4.8 Transfer Learning & Feature Extraction

Transfer learning is the practice of reusing knowledge learned on one task for a different (but related) task. In this notebook, a CNN trained for image classification on CIFAR-10 is used as a feature extractor: the final classification layer is discarded, and the 84-dimensional activations from fc2 are used as rich, semantically meaningful feature vectors.
The CNN has learned to compress a 3072-dimensional raw image (32x32x3) into an 84-dimensional representation that captures visually relevant structure. This representation is far more useful for downstream classifiers than raw pixels, which is why the Random Forest achieves better accuracy on CNN features than it would on raw pixels.

4.9 Normalization (Data Preprocessing)

Normalizing input data to zero mean and unit variance is critical for stable training. Without normalization, features with large magnitudes dominate gradient updates, learning rates become hard to tune, and activation functions may saturate. The transform Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) shifts each RGB channel from [0,1] to [-1,1], centering the input distribution and improving gradient flow.

4.10 Batch Processing

Rather than processing one sample at a time (online learning) or the entire dataset at once (batch learning), mini-batch gradient descent uses small batches (here, 4 images). This balances the benefits of both: more frequent updates than batch learning, but smoother gradients than online learning. Modern GPUs are highly parallelized and compute an entire batch as efficiently as a single sample, making batching essential for practical training speed.

4.11 CIFAR-10 Dataset

CIFAR-10 (Canadian Institute for Advanced Research) is a benchmark dataset consisting of 60,000 color images (32x32 pixels) across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. It is split into 50,000 training and 10,000 test images. CIFAR-10 is widely used to benchmark image classification algorithms because it is large enough to be challenging but small enough to train on consumer hardware.

4.12 Random Forest

Random Forest is an ensemble learning method that operates by constructing many decision trees during training and outputting the mode of the classes (for classification) from individual trees. Each tree is built on a bootstrap sample of the training data (bagging) and at each node, only a random subset of features is considered for splitting (feature randomness). Together,

these two sources of randomness reduce variance and improve generalization over a single decision tree.
In this notebook, 100 trees vote on the class of each test image's 84-dimensional CNN feature vector. The combination of CNN feature extraction and Random Forest classification often outperforms either approach alone.

4.13 Feature Importance

Random Forest provides a built-in measure of feature importance based on the mean impurity decrease (Gini importance) contributed by each feature across all trees. A feature is considered important if splitting on it tends to produce purer child nodes. In this context, clf.feature_importances_ reveals which of the 84 CNN neurons are most discriminative for CIFAR-10 classification — providing interpretability into what the CNN has learned.

4.14 Overfitting and Generalization

Overfitting occurs when a model memorizes training data rather than learning generalizable patterns. Signs include high training accuracy but low test accuracy. Techniques to combat overfitting include: Dropout (randomly zeroing activations during training), data augmentation (applying random transforms to training images), weight decay (L2 regularization), and batch normalization. This notebook does not use these techniques in the CNN, which is why training for more epochs would likely cause overfitting.

4.15 Epoch

One epoch is a complete pass through the entire training dataset. The notebook trains for 1 epoch, meaning each training image is seen exactly once. In practice, multiple epochs are needed for the model to converge. With 50,000 training images and batch size 4, one epoch consists of 12,500 gradient update steps.

4.16 GPU Acceleration (CUDA)

GPUs (Graphics Processing Units) are massively parallel processors with thousands of small cores optimized for the matrix operations that dominate neural network computation. CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform that allows PyTorch to run tensor operations on GPU, typically achieving 10-100x speedup over CPU for deep learning workloads. PyTorch abstracts CUDA through .cuda() and .to('cuda') methods.

4.17 Softmax and Logits

The final layer of the CNN produces 10 raw scores (logits) — one per class. Softmax converts these into a probability distribution summing to 1: p_i = exp(score_i) / sum(exp(score_j)). The class with the highest probability (or equivalently, the highest logit — since softmax is monotone) is the prediction. PyTorch's CrossEntropyLoss applies softmax internally, so the forward pass returns raw logits.

4.18 Model Persistence

In practice, training a neural network is expensive, so trained weights are saved to disk (model persistence) for later inference or fine-tuning. PyTorch saves the state_dict (weight tensors only) rather than the full model object, which is more portable and version-stable. Scikit-learn models are saved via joblib, which handles the NumPy arrays inside Random Forest objects more efficiently than plain pickle.

5. CNN Architecture Summary

The following table summarizes the layer-by-layer dimensions through the Net architecture for a single 32x32 RGB input image:

Layer	Operation	Input Shape	Output Shape	Parameters	Notes
conv1	Conv2d(3,6,5)	(3,32,32)	(6,28,28)	456	3*5*5*6 + 6
relu1	ReLU	(6,28,28)	(6,28,28)	0	
pool1	MaxPool2d(2,2)	(6,28,28)	(6,14,14)	0	
conv2	Conv2d(6,16,5)	(6,14,14)	(16,10,10)	2,416	6*5*5*16 + 16
relu2	ReLU	(16,10,10)	(16,10,10)	0	
pool2	MaxPool2d(2,2)	(16,10,10)	(16,5,5)	0	
flatten	view(-1,400)	(16,5,5)	(400,)	0	x3
fc1	Linear(400,120)	(400,)	(120,)	48,120	
relu3	ReLU	(120,)	(120,)	0	
fc2	Linear(120,84)	(120,)	(84,)	10,164	x2 — feature vector
relu4	ReLU	(84,)	(84,)	0	
fc3	Linear(84,10)	(84,)	(10,)	850	Class logits

Total trainable parameters: approximately 61,006

6. Quick Glossary

Tensor: Multi-dimensional array; PyTorch's primary data structure. Similar to NumPy ndarray but with GPU support and autograd.

Autograd: PyTorch's automatic differentiation engine. Tracks operations on tensors and computes gradients automatically.

Epoch: One complete pass through the training dataset.

Batch: A subset of training samples processed together in a single forward/backward pass.

Gradient: The partial derivative of the loss with respect to a parameter; indicates direction of steepest ascent.

Weight: A learnable parameter in a neural network layer, updated during training.

Logit: Raw, unnormalized score output by the final layer before softmax.

state_dict: An OrderedDict mapping layer names to their parameter tensors; used for model saving/loading.

Bootstrap: Sampling with replacement; used in Random Forest to create diverse training sets for each tree.

Impurity: A measure of class mixing in a decision tree node; lower impurity means a purer (more confident) split.